⚠️ **TRAINING EXAMPLE — NOT A REAL IEC 62304 DELIVERABLE**

This project is a **training site, not a real medical device** under any regulatory definition. The device described in this document — "SentinelFlow 500" — is entirely **fictional**, invented for this course: nothing here is a genuine IEC 62304 deliverable, a real regulatory submission, or evidence of any real organisation's compliance with anything. It illustrates the *kind* of document a real SaMD/SiMD project would produce. See the document register for the full explanation.

Clause: 4 — General Requirements · **Register:** Device Example Register

Document ID: SOP-01 · **Device:** SentinelFlow 500 (fictional) · **Safety class:** C
Status: SOP — worked example

> **Fictional worked example.** The "SentinelFlow 500" ambulatory infusion pump described in this document does not exist. It was invented for this course so a reader could see an IEC 62304 document written the way a real QA/RA-controlled procedure reads — Purpose, Scope, Responsibilities, Procedure, Records — about an actual medical device, rather than as a teaching explanation of the clause. See the Device Example Register for the device description this SOP and the rest of the set refer back to, and the self-referential set for the parallel version of this document that is genuinely true, because it is about this training website's own (non-medical) development instead.

1. Purpose

This SOP defines how SentinelFlow 500's software development integrates with the quality management system, how software risk activities connect to the device risk management process, how the safety class of each software item is determined and recorded, and how legacy software would be assessed if it were ever used. It exists to satisfy IEC 62304 Clause 4, which every other SOP in this set (SOP-02 through SOP-13) assumes is already in place — this is the document a new software team member reads first.

2. Scope

Applies to all software development, maintenance, and risk management activities for SentinelFlow 500's embedded control software — see the device description for what that software does. Covers Clause 4 in full: quality management system context (§4.1), risk management integration (§4.2), software safety classification (§4.3), and legacy software (§4.4). Does not itself define *how* risk management, planning, or classification are carried out step by step — those procedures live in their own SOPs, referenced in §5 below; this document defines the context they all operate inside.

3. Responsibilities

ROLE	RESPONSIBILITY UNDER THIS SOP
Software Development Lead	Owns this SOP's execution for a given project; ensures every software item is classified before development activities begin
QA/RA	Reviews and approves the classification rationale; maintains the QMS this SOP operates under; approves any legacy software gap analysis
Risk Manager	Owns the device-level ISO 14971 risk management file; agrees the hazardous-situation analysis that §4.3 classification depends on

4. Definitions & Abbreviations

TERM	MEANING
QMS	Quality management system (ISO 13485)
Hazardous situation	Circumstance in which a person is exposed to a hazard (per IEC 62304 Amendment 1 terminology)
Legacy software	Software already placed on the market for which there is insufficient objective evidence it was developed to this standard — a test of missing evidence, not age

TERM	MEANING
Class A / B / C	Software safety class, per §4.3 — see §7.3 below for how it is determined

5. References

- IEC 62304:2006+AMD1:2015, Clause 4
- ISO 13485 — the QMS route this SOP assumes (§4.1)
- ISO 14971 — the risk management process this SOP hands off to (§4.2)
- **SOP-11 — Software Risk Management File** (Clause 7 device example, partial worked example): the device-level hazard analysis and risk control measures that §4.3's classification rationale in §7.3 below is drawn from
- **SOP-02 — Software Development Plan** (Clause 5.1 device example, template): the plan this SOP's QMS and risk-management context feeds into

6. What the standard requires

REF	TITLE	APPLIES AT CLASS C	WHAT MUST BE PRODUCED
4.1	Quality management system	Yes	Satisfied by pointing at a QMS — see §7.1.
4.2	Risk management	Yes	Satisfied by pointing at ISO 14971 — see §7.2.
4.3	Software safety classification	Yes	Software safety class of each software system and software item, recorded in the risk management file, with a rationale where an item is classified differently from its parent.

REF	TITLE	APPLIES AT CLASS C	WHAT MUST BE PRODUCED
4.4	Legacy software	Yes	Gap analysis against 5.2, 5.3, 5.7 and Clause 7; a plan to close the gaps, with software system test records as the minimum deliverable; and the legacy software version plus a documented rationale for its continued use.

(Quoted directly from `applicability.json` — Clause 4 carries no per-requirement `[Class ...]` tags in the standard itself; Table A.1 assigns all four sub-clauses to every class. This table is identical to the one in the self-referential set's 01 — the requirements don't change with the device, only the answers do.)

7. Procedure

7.1 Quality management system (§4.1)

SentinelFlow 500 is manufactured under a certified ISO 13485 QMS, as any real infusion pump manufacturer would be required to have. This document set does not — and cannot — demonstrate that QMS in operation, because there is no real organisation behind a fictional product to audit; that is stated here as an assumption the rest of this SOP set relies on, not as evidence being claimed.

7.2 Risk management (§4.2)

The normative text names ISO 14971 as the single route, with no alternative offered — unlike §4.1's three routes. Software risk activities for SentinelFlow 500 are not run as a separate process: every hazardous situation the software could contribute to is recorded in the same risk management file the device team uses for hardware and use-related risks. **SOP-11 — Software Risk Management File** owns that file; this SOP only draws on its output — see §7.3 below.

7.3 Software safety classification (§4.3)

The test, as Amendment 1 states it: a software item that could contribute to a hazardous situation must be Class B minimum; if it is the sole control measure preventing serious injury or death, it must be Class C. Software items that cannot contribute to any hazardous situation may be Class A.

Applying it to SentinelFlow 500's control software: the software directly controls a motor delivering medication into a patient's bloodstream. Consider three credible failure modes:

- **Undetected occlusion.** A blocked line stops delivery; if the occlusion sensor's software path fails to raise the alarm, a time-critical drug (e.g. a vasoactive infusion) simply stops reaching the patient with no warning.
- **Free-flow over-infusion.** If the software fails to keep the motor stopped when it should be closed (e.g. during a pause, or while the administration set is being changed), gravity can free-flow the full reservoir into the patient far faster than programmed — a classic, well-documented infusion pump hazard.
- **Misprogrammed or misapplied rate.** If the software's parameter validation fails to catch a rate outside the configured safe range for the selected drug profile, an order-of-magnitude dosing error can be delivered exactly as "successfully" as a correct one.

In each case the harm (death or serious injury, depending on the drug) is severe, and — critically — **the software is the sole control measure**: there is no independent mechanical flow limiter or hardware interlock downstream of it that would catch these failures before they reach the patient. That is exactly the condition the test names for Class C, not merely Class B.

Result: Class C for the delivery-control software item, recorded in the risk management file (SOP-11) as required by the Output column in §6 above. Where a future software item is split out of the control software with a narrower function that cannot itself contribute to a hazardous situation (e.g. a read-only usage-statistics logger), it would be classified separately and could legitimately land at Class A — §4.3's requirement to record "a rationale where an item is classified differently from its parent" exists for exactly that case.

7.4 Legacy software (§4.4)

When this procedure applies: before any software item is brought into a project without being developed under this SOP set from the outset — whether acquired, inherited from a prior product line, or carried over from an earlier, less rigorously documented version — the Software Development Lead must run a gap analysis against §5.2 (requirements), §5.3 (architecture), §5.7 (system testing) and Clause 7 (risk management), produce a plan to close whatever gaps that analysis finds (with software system test records as the minimum deliverable), and record the software's version and a rationale for continuing to use it.

Applied to SentinelFlow 500: not applicable. SentinelFlow 500 is presented as developed from a single IEC 62304 lifecycle from inception, with no prior undocumented version on the market — so there is no legacy software gap analysis to perform for this release. Stated as such rather than left blank, per this document set's convention for genuinely inapplicable sub-clauses.

8. Records

RECORD	PRODUCED BY	RETAINED IN
Software safety classification and rationale (§7.3)	Software Development Lead, reviewed by QA/RA	Risk management file (SOP-11)
Legacy software gap analysis (§7.4), if triggered	Software Development Lead	Project technical file

9. Revision History

Illustrative only — SentinelFlow 500 has no real revision history.

VERSION	STATUS	DESCRIPTION
1.0	Effective	Initial issue